Home Control

Home automation server with X10/Mochad relay control, MQTT integration, and sunrise/sunset scheduling.

What it does

Controls lights, sprinklers, and other home automation devices through a web interface. Supports three device groups:

- **Lights A** — X10 RF modules via Mochad (CM15A/CM19A controller)
- **Lights B** — MQTT-connected devices (Tasmota smart switches/plugs)
- **Sprinklers** — MQTT-controlled irrigation zones with duration timers

Each group has a **Configuration** page:

Page	URL	Where the timers live
Lights A	`/config_lights_a`	`config.json` → compiled into `at` jobs
Lights B	`/config_lights`	each light's own Tasmota device (own IP/MQTT topic)
Sprinklers	`/config`	the sprinkler Tasmota device (start time + duration + days)

How it works

```
Web Browser ──HTTP──▶ Flask Server ──MQTT──▶ Tasmota devices (Lights B, sprinklers)
                                   ├──X10/Mochad──▶ RF modules (Lights A)
                                   └──at jobs──▶ scheduled RF events
```

Key components

File	Purpose
`hc_server.py`	Flask web server, MQTT client, X10 relay, all three config pages
`config.json`	Devices, MQTT broker, location, Lights A schedule
`update_crontab.py`	Compiles the `config.json` schedule into `at` jobs
`x10_mochad_relay_asyncio.py`	Optional X10 RX→TX relay via the Mochad TCP socket
`static/style.css`	Web UI styling (dark mode)
`templates/*.html`	Home page + per-group configuration pages
`hc_server.service` / `hc_server.timer`	systemd units for the web server
`update_crontab.service` / `.timer`	systemd units for the daily schedule rebuild
`ruff.toml`	Lint configuration used by the test suite

Installation

Prerequisites

```
sudo apt-get install python3 python3-pip mosquitto mosquitto-clients netcat-openbsd at
pip install flask paho-mqtt suntime ruff
```

Mochad (X10 controller)

Install mochad — the RF-to-Ethernet bridge for CM15A/CM19A:

```
wget https://github.com/jrh20001/mochad-0.1.17/archive/refs/tags/0.1.17.tar.gz
tar xzf 0.1.17.tar.gz && cd mochad-0.1.17
./configure && make && sudo make install
sudo mochad
```

Configuration

Edit `config.json`:

```
{
  "mqtt": {
    "broker": "hcontrol.local",
    "port": 1883,
    "topic_cmnd": "cmnd/tasmota_sprinklers/",
    "topic_stat": "stat/tasmota_sprinklers/RESULT"
  },
  "devices": {
    "a1": { "name": "Kitchen", "state": "off", "group": "1", "swno": "1", "dimr": "0" },
    "outdoor_side": { "name": "Garage side", "state": "off", "group": "2", "swno": "1", "dimr":
    "spare8": { "name": "Spare8", "state": "off", "group": "3", "swno": "8", "dimr": "0" }
  },
  "location": { "latitude": 37.37006, "longitude": -122.08839 },
  "schedule": [
    { "time": "sunset", "offset": "00:00:00", "device": "a1", "action": "on" },
    { "time": "today", "offset": "22:29:00", "device": "a1", "action": "off" }
  ]
}
```

Device groups: - "1" — Lights A (X10 RF via Mochad; swno unused for RF) - "2" — Lights B (MQTT/Tasmota; swno selects the relay) - "3" — Sprinklers (MQTT/Tasmota; swno selects the relay)

Schedule entries: `time` ∈ `today | tomorrow | sunrise | sunset`, `offset` = ±`HH:MM:SS` (hours/minutes move the time; the **seconds are a stagger sleep** before the RF command).

Running

```
python3 hc_server.py          # web server, binds 0.0.0.0:80
python3 update_crontab.py     # (re)build the at-job queue from the schedule
```

Then open `http://hcontrol.local/` (all pages, including the configuration pages, are plain HTML forms).

systemd

```
sudo cp hc_server.service hc_server.timer /etc/systemd/system/
sudo cp update_crontab.service update_crontab.timer /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now hc_server.service hc_server.timer update_crontab.timer
```

`hc_server.service` runs as the unprivileged `ubuntu` user and is granted only `CAP_NET_BIND_SERVICE` (needed to bind port 80). The schedule rebuild runs as the same user so the `at` jobs it manages stay visible to it.

Usage

1. Open `http://hcontrol.local/`
2. Toggle devices from the group pages (`/<device>/<on|off>`)
3. Use **Configuration** at the bottom of a group page:
   - **Lights A**: edit the `config.json` schedule (add/delete/edit entries, live “next run” preview). Saving rewrites `config.json` atomically with a timestamped backup, hot-reloads the server, and regenerates the `at` queue.
   - **Lights B**: per light `On`/`Off` timers with mode (`At time` / `Sunrise` / `Sunset`), sign + offset, shared day row and per-light master switch. Values are written to each Tasmota device.
   - **Sprinklers**: start time, duration and days for six zones; the end timer is derived from start + duration.

Endpoints

- `GET /` — home page (Lights A)

- `GET /<device>/<on|off>` — toggle a device and show its group page
- `GET|POST /config` — sprinkler timers
- `GET|POST /config_lights` — Lights B timers
- `GET|POST /config_lights_a` — Lights A schedule (edits `config.json`)

Tests and lint

```
cd <a writable copy of this directory>      # hc_server writes hc_server.log in the module dir
python3 -m unittest discover -p 'test_*.py' -v
ruff check .
```

`test_*.py` covers the sprinkler/Lights A/Lights B configuration paths, MQTT payload parsing (flat and legacy chunked Tasmota replies), `at`-job generation (including past-time rollover and unrelated-job preservation) and route handling.

Notes

- `config.json` is the only place Lights A timers are stored; Lights B and sprinkler timers live on the Tasmota devices.
- `light_ip_cache.json` (runtime cache of light IP addresses) and `*.log` files are not tracked.
- Unreachable devices are shown with their last known values and are skipped when saving.